

MY DATA WRANGLING PROJECT WITH SQL (MYSQL)



Data wrangling-also called data cleaning, data remediation, or data munging refers to a variety of processes designed to transform raw data into more readily used formats. This is a prerequisite for a successful data analysis; it helps one to

properly and efficiently analyze data as to make good business decision.

In this Project, I will walk you through every single step I took to remediate this Significant Earthquake dataset. It was downloaded from Kaggle; This dataset includes a record of the date, time, location, depth, magnitude, and source of every earthquake with a reported magnitude 5.5 or higher since 1965 to 2016. It has over 23K+ rows and 21 columns.

Aside my personal reason for choosing this dataset (curiosity about natural disaster), the second reason is to demonstrate my ability to work on wrongly formatted variables (most especially dates and time) to get a well structured dataset that will pose no error during any analysis process.

Before importing the csv dataset into MySQL workbench: I noticed that some data in the date column were left aligned;

A	
Date	T
01/02/1965	
01/04/1965	
01/05/1965	
01/08/1965	
01/09/1965	
01/10/1965	
01/12/1965	
01/15/1965	
01/16/1965	
01/17/1965	
01/17/1965	
01/24/1965	
01/29/1965	
02/01/1965	
02/02/1965	
02/04/1965	
02/04/1965	

Date format in excel after download

I solved this misalignment by making sure the date column were formatted using text to column. This I did by having a closer look

on the Date column. I noticed that Excel treated it as 'DMY' instead of 'MDY'; owing to the fact it treated the cells that their supposed month value is greater than 12 as 'text' because it could not interpret it as date since the highest numerical value for month is 12.

Steps includes:

Text to column>>Delimited>>Others>>Date(Choose option MDY)>> Click Finish

A		B	
Date		Date2	Tim
	01/02/1965		02/01/1965
	01/04/1965		04/01/1965
	01/05/1965		05/01/1965
	01/08/1965		08/01/1965
	01/09/1965		09/01/1965
	01/10/1965		10/01/1965
	01/12/1965		12/01/1965
	01/15/1965		15/01/1965
	01/16/1965		16/01/1965
	01/17/1965		17/01/1965
	01/17/1965		17/01/1965
	01/24/1965		24/01/1965
	01/29/1965		29/01/1965
	02/01/1965		01/02/1965
	02/02/1965		02/02/1965
	02/04/1965		04/02/1965
	02/04/1965		04/02/1965
	02/04/1965		04/02/1965
	02/04/1965		04/02/1965
	02/04/1965		04/02/1965
	02/04/1965		04/02/1965

Date format after using text to column

After which I imported them into MySQL server using the MySQL Table import wizard.

The steps I took for this data wrangling project in MYSQL includes Data Discovery, Data structuring and Cleaning which includes Reformatting of some wrongly formatted data types, Checking and handling data entry inconsistency/irregularities, handling blank values, checking for duplicates and so on.

LETS GET STARTED!!!

I used MySQL Server to develop this project.

DATA DISCOVERY:

```
SELECT * from datacleaning.earthquakes;
```

Press enter or click to view image in full size

	Date	Time	Latitude	Longitude	Type	Depth	Depth Error	Depth Seismic Stations	Magnitude	Magnitude Type
►	01/02/1965	13:44:18	19.246	145.616	Earthquake	131.6			6	MW
	01/04/1965	11:29:49	1.863	127.352	Earthquake	80			5.8	MW
	01/05/1965	18:05:58	-20.579	-173.972	Earthquake	20			6.2	MW
	01/08/1965	18:49:43	-59.076	-23.557	Earthquake	15			5.8	MW
	01/09/1965	13:32:50	11.938	126.427	Earthquake	15			5.8	MW

A glance at my dataset, shows that it contains date, time, text, blank values and some numerical values.

In order to make it better organized for easier consumption and analysis, I checked the format/datatypes of the various variables as to configure each of them so they can be understood and examined to find patterns and trends during analysis.

```
SELECT DATA_TYPE from INFORMATION_SCHEMA.COLUMNS where  
table_schema = 'datacleaning' and table_name = 'earthquakes';
```

Press enter or click to view image in full size

Result Grid	
	DATA_TYPE
▶	text
	text
	double
	double
	text
	double
	text
	text
	double
	text
	text
	text
	text
	text
	text

This shows that date, time and some numerical values were stored as text datatype; These will definitely throw up errors during calculation and will also bring out wrong query results that will skew our analysis and present a wrong insight.

To fix these ;I moved to the next step:

DATA STRUCTURING & CLEANING:

My earthquake table shows a text-laden values that contains Date, Time etc. that needs to be parsed.

Checking for Data Entry Inconsistency/irregularities:

Before converting the Columns into the right data type; I checked and handled them for some inconsistency:

This I did with the **LENGTH** and **LEFT** functions. The **LENGTH** function was used to return the length of the date columns also checking for any lesser or greater value while the **LEFT** function was used to calculate a new value that is consistence along all entries.

For Date column: To be sure that the length of the values are all the same in the columns.

Press enter or click to view image in full size

```
-- Handling data entry inconsistency
SELECT length(date), MAX(length(date)), min(length(date))
from datacleaning.earthquakes;-- min=10, Max=24

-- Making sure that there are no other length apart from the two above
SELECT date from datacleaning.earthquakes
where length(date) != 10 AND length(date) != 24; -- 0

select count(date) from datacleaning.earthquakes
where length(date) = 24 -- 3
```

This shows that there are varying length of dates (10 and 24) of which 3 out of the whole rows have length of 24. This was fixed with the LEFT function.

Press enter or click to view image in full size

```
SELECT LEFT(Date,10) from datacleaning.earthquakes;

UPDATE datacleaning.earthquakes SET Date = LEFT(date,10);-- 3 rows were affected

Select Date from datacleaning.earthquakes where length(Date) = 24; -- 0 rows
```

To Standardize the Date column:

Now that all the columns have the same length (10). To convert the Date ***‘text datatype’*** to ***‘date datatype’***; A new column **Date2** was created and the STR_TO_DATE function will be used to convert the initial values from text to date.

Press enter or click to view image in full size

```
ALTER TABLE datacleaning.earthquakes
ADD column Date2 date after Date;

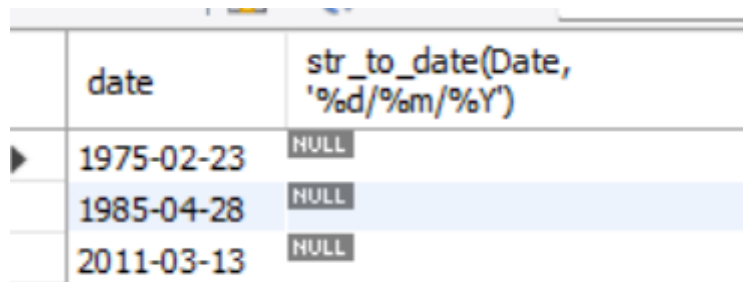
UPDATE datacleaning.earthquakes
SET Date2 = STR_TO_DATE(Date, '%d/%m/%Y'); -- throws up some error of incorrect datetime values
-- To find the cause of the error
Select date, str_to_date(Date, '%d/%m/%Y') from datacleaning.earthquakes
where str_to_date(Date, '%d/%m/%Y') is null;

-- 3 of them (since there are not much I will manually update the 3 columns with REPLACE fnx
```

This Query shows some null values in the result and a failed text result of ‘incorrect datetime value’ for some rows; This was as a result of some wrong inputted date values as I confirmed with this syntax

***Select date, str_to_date(Date, '%d/%m/%Y') from
datacleaning.earthquakes
where str_to_date(Date, '%d/%m/%Y') is null;***

which results to the below snippet



	date	str_to_date(Date, '%d/%m/%Y')
▶	1975-02-23	NULL
	1985-04-28	NULL
	2011-03-13	NULL

This query shows that I have 3 irregular date entry format that is different from the rest that has this format “01/02/1965”. These I changed with the **UPDATE** and **REPLACE** function.

Press enter or click to view image in full size

```

update datacleaning.earthquakes
SET Date = Replace(Date, '1975-02-23', '23/02/1975');

update datacleaning.earthquakes
SET Date = Replace(Date, '1985-04-28', '28/04/1985');

update datacleaning.earthquakes
SET Date = Replace(Date, '2011-03-13', '13/03/2011');

Select date, str_to_date(Date, '%d/%m/%Y') from datacleaning.earthquakes
where str_to_date(Date, '%d/%m/%Y') is null;-- 0 [Error has been corrected]

```

After all these processes; **I have to standardize the Date column again**

Press enter or click to view image in full size

```

-- To update the new Date2 column again
UPDATE datacleaning.earthquakes
SET Date2 = STR_TO_DATE(Date, '%d/%m/%Y');-- 23412 rows were affected

select date, date2 from datacleaning.earthquakes;

```

Press enter or click to view image in full size

12 14:33:40 UPDATE datacleaning.earthquakes SET Date2 = STR_TO_DATE(Date, '%d/%m/%Y') 23412 row(s) affected Rows matched: 23412 Changed: 23412 Warnings: 0

	Date	Date2
▶	02/01/1965	1965-01-02
	04/01/1965	1965-01-04
	05/01/1965	1965-01-05
	08/01/1965	1965-01-08
	09/01/1965	1965-01-09

Now I have a standardized column **Date2** that could be use to do further analysis based on date in Tableau, Power BI, etc.

For the Time Column:

A new column **Time2** was created and the **CAST** function will be used to convert the initial values from text to time.

Press enter or click to view image in full size

```
-- To standardize the time column
select cast(Time as time) from datacleaning.earthquakes;

Alter table datacleaning.earthquakes add Time2 time after Time;

Update datacleaning.earthquakes
set Time2 = cast(Time as time);-- this threw up error ' Truncated incorrect time value
```

The **Update** function for Time threw up error which I traced with the LENGTH function; which shows that Time column have varying string length of 8 and 24, of which 3 rows out of the whole 23k+ rows are having the length 24.

Press enter or click to view image in full size

```
-- To show the abnormal time length:  
select Time from datacleaning.earthquakes where length(Time) = 24;
```

Result Grid |   Filter Rows:

	time
	1975-02-23T02:58:41.000Z
	1985-04-28T02:53:41.530Z
	2011-03-13T02:23:34.520Z

They were cleaned and updated accordingly in **Time** column before Updating the new column **Time2** using the UPDATE , REPLACE and SUBSTR function:

Press enter or click to view image in full size

```
-- To replace the 3 rows with the correct Time length
update datacleaning.earthquakes
Set Time = replace(Time, '1975-02-23T02:58:41.000Z', substr(Time,12,8))
where Time = '1975-02-23T02:58:41.000Z';

update datacleaning.earthquakes
Set Time = replace(Time, '1985-04-28T02:53:41.530Z', substr(Time,12,8))
where Time = '1985-04-28T02:53:41.530Z';

update datacleaning.earthquakes
Set Time = replace(Time, '2011-03-13T02:23:34.520Z', substr(Time,12,8))
where Time = '2011-03-13T02:23:34.520Z';
```

To finally Update the new column Time2:

```
Update datacleaning.earthquakes
set Time2 = cast(Time as time);

select Time, Time2 from datacleaning.earthquakes;
```

CHECKING AND HANDLING OF BLANK VALUES:

The blank values in this case means no value was recorded for these particular dates which I changed to “o”.These were done

for all the supposed numerical values in the dataset before

converting them to a numerical datatype

I used the count ,**CASE** and REPLACE functions ; e.g.,

Press enter or click to view image in full size

```
-- Using CASE FNX handle the blank values in the columns before converting them
-- to the appropriate datatype
▶ Select count(Depth_Error) from datacleaning.earthquakes where Depth_Error = '';-- 18951

▶ update datacleaning.earthquakes
  Set Depth_Error = case
    when Depth_Error = '' then 0.0
    else Depth_Error
  end;
```

The count function is to help me know as well make sure that its that exact number of rows that was affected.

Converting the numerical data that were stored as text to double:

The Alter and Modify column function were used;

Press enter or click to view image in full size

```
-- Converting the numerical data that were stored as text to double

ALTER TABLE earthquakes Modify column Depth_Error double;
ALTER TABLE earthquakes Modify column Depth_Seismic_Stations double;
ALTER TABLE earthquakes Modify column Magnitude_Error double;
ALTER TABLE earthquakes Modify column Magnitude_Seismic_Stations double;
ALTER TABLE earthquakes Modify column Azimuthal_Gap double;
ALTER TABLE earthquakes Modify column Horizontal_Distance double;
ALTER TABLE earthquakes Modify column Horizontal_Error double;
ALTER TABLE earthquakes Modify column Root_Mean_Square double;
```

These leaves me with well formatted columns that can be used for calculations without any fear of misinformation.

Checking for Duplicates using CTE and ROW NUM:

A common table expression, or CTE, (in [SQL](#)) is a temporary named result set, derived from a simple query and defined within the execution scope of a `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement.(Wikipedia)

while ROW_NUMBER (ROW_NUM) function is a SQL ranking function that assigns a sequential rank number to each new record in a partition. This assigns a unique number to each row in my dataset and shows the number of times a row with the same data appears in the dataset.

Once I defined the CTE, it will be easy to delete the rows that appeared more than 1(i.e. duplicates),

Press enter or click to view image in full size

```
-- CHECKING FOR DUPLICATES USING CTE
With t1 as (
SELECT *, row_number() over(partition by Date2, Time2, Latitude, Longitude order by ID) rownum
from datacleaning.earthquakes)
Select count(*) from t1 where rownum >1; -- 0 (No duplicate values)
```

This shows that there are no duplicate row.

**CREATING NEW COLUMNS (YEAR, MONTH, DAY,
WEEK, DAY OF WEEK) FROM THE DATE2 COLUMN**

I used the **EXTRACT** function to get these values from the formatted date column Date2.

Year:

Press enter or click to view image in full size

```
-- Year
SELECT EXTRACT(YEAR FROM Date2) from datacleaning.earthquakes;

alter table datacleaning.earthquakes add column Year int after Time2;

Update datacleaning.earthquakes set Year =
extract(YEAR FROM Date2);
```

Month:

Press enter or click to view image in full size

```
select extract(MONTH FROM Date2) from earthquakes;
alter table datacleaning.earthquakes add column Month int after Year;

Update datacleaning.earthquakes set Month =
extract(Month FROM Date2);
```

Week, Week Name and Day name:

Press enter or click to view image in full size

```
Select week(Date2,0) from earthquakes;
alter table datacleaning.earthquakes add column Week int after Month;
update datacleaning.earthquakes set Week =
week(Date2,0);
-- Day of the week
select dayname(Date2) from earthquakes;
alter table datacleaning.earthquakes add column Weekdays character after Week;

update earthquakes set Weekdays =
dayname(Date2); -- This throws up Error 1406. Data too long for column

alter table datacleaning.earthquakes modify column Weekdays character (15);
update earthquakes set Weekdays =
dayname(Date2);
```

CHECKING FOR OUTLIERS:

An outlier is an observation that lies an abnormal distance from other values in a random sample from a population.

For my dataset; **These data where collected within the year 1965 to 2016 and for only earthquakes with magnitude of 5.5 and above.** So any date that is above or

beyond these ranges is an outlier in this case as well as any
Magnitude lesser than 5.5.

Press enter or click to view image in full size

```
-- Looking for outliers (with the knowledge that the years data were collected were 1965-2016  
-- and magnitude is >= 5.5)  
select year from earthquakes  
where Year < 1965 or Year > 2016;  
  
select * from earthquakes  
where Magnitude < 5.5; -- 0
```

The query shows no outliers. If peradventure, I had encountered one, I would delete it in this case; since I have a particular target range for this analysis.

DROPPING OF UNUSED COLUMNS:

Date and Time columns that we recreated are not necessary again as I have new column that have the same data in a more organized and formatted form.

```
-- Deleting UNUSED COLUMN
Alter table datacleaning.earthquakes
Drop column Date,
Drop column Time ;

Select * from datacleaning.earthquakes;
```

Press enter or click to view image in full size

	Date2	Time2	Year	Month	Week	Weekdays	Latitude	Longitude	Type	Depth	Depth_Error	Depth
▶	1965-01-02	13:44:18	1965	1	0	Saturday	19.246	145.616	Earthquake	131.6	0	0
	1965-01-04	11:29:49	1965	1	1	Monday	1.863	127.352	Earthquake	80	0	0
	1965-01-05	18:05:58	1965	1	1	Tuesday	-20.579	-173.972	Earthquake	20	0	0
	1965-01-08	18:49:43	1965	1	1	Friday	-59.076	-23.557	Earthquake	15	0	0
	1965-01-09	13:32:50	1965	1	1	Saturday	11.938	126.427	Earthquake	15	0	0
	1965-01-10	13:36:32	1965	1	2	Sunday	-13.405	166.629	Earthquake	35	0	0

CONCLUSION: I deleted the unused columns because the information are contained in my new created columns and I also have a duplicate of the old unformatted data.

Now I have a well wrangled data that is well formatted and can be used for further analysis without any fear and most especially; I was able to show the ability to parsed Date and Time, Detect

and correct badly formatted data and check for other error that can skew my analysis.